

Day 5 — Logistic Regression

Objective

Understand logistic regression as a probabilistic classification model, not just a sklearn class.

Topics and detailed notes

- A linear score is transformed into a probability using the sigmoid.
- Probability is continuous; class prediction is obtained by applying a decision threshold.
- Log-odds transform probability into an unbounded linear quantity.
- Binary cross-entropy strongly penalizes confident wrong predictions.
- Regularization controls model complexity.
- Multiclass classification can be handled with appropriate multiclass strategies.
- Logistic regression is a strong baseline because it is fast, relatively interpretable, and produces probabilities.

Mathematics / formulas to know

1. Gini Impurity

$$Gini = 1 - \sum_{k=1}^K p_k^2$$

Binary case:

$$Gini = 1 - (p_0^2 + p_1^2)$$

2. Entropy

$$H = - \sum_{k=1}^K p_k \log_2(p_k)$$

For binary classification:

$$H = -p_0 \log_2(p_0) - p_1 \log_2(p_1)$$

3. Information Gain

$$IG = H(\text{parent}) - \sum_j \frac{n_j}{n} H(\text{child}_j)$$

For two children:

$$IG = H(\text{parent}) - \left[\frac{n_L}{n} H(L) + \frac{n_R}{n} H(R) \right]$$

The tree wants to maximize information gain.

4. Weighted Gini after splitting

$$Gini_{split} = \frac{n_L}{n} Gini_L + \frac{n_R}{n} Gini_R$$

The tree wants a lower resulting impurity.

5. Regression Tree — MSE

For a node:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2$$

A regression tree searches for splits that reduce the prediction error/variance inside the resulting nodes.

6. Regression Tree — variance

$$Var(Y) = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2$$

7. Tree complexity

You should understand the relationship between:

Tree depth $\rightarrow$ model complexity $\rightarrow$ bias/variance

Increasing tree depth generally:

bias $\downarrow$

but can cause:

variance $\uparrow$

which can lead to overfitting.

Implementation pattern

X/y $\rightarrow$ split $\rightarrow$ preprocessing $\rightarrow$ baseline $\rightarrow$ fit $\rightarrow$ probability predictions $\rightarrow$ threshold $\rightarrow$ class predictions $\rightarrow$ metrics $\rightarrow$ error analysis.

Experiments to perform

- Change the classification threshold without retraining.
- Compare probability outputs with hard class outputs.
- Implement sigmoid and log loss with NumPy.
- Inspect coefficients and their direction.

End-of-day deliverables

- Customer churn prediction.
- Probability-vs-threshold experiment.
- Confusion matrix at multiple thresholds.

Mastery questions

1. Why is 0.5 not always the correct threshold?
2. What does a probability of 0.8 mean operationally?
3. Why is log loss useful even when classification accuracy is high?

PLANE

Welcome to Day 5. You have already survived the steepest learning curves of system architecture, and today we are looking at another classic syllabus plot twist.

Just like Day 4, your Day 5 curriculum has a split personality. The textual objectives strictly focus on **Logistic Regression** as a probabilistic baseline, but the hidden mathematical formulas provided in your document are the exact blueprints for building **Decision Trees**!

As a Senior Engineer, we do not panic; we build both. We are going to master the probabilistic business logic of Logistic Regression, and then we will crack open the math behind Decision Tree algorithms.

Phase 1: Logistic Regression (The Probabilistic Baseline)

You already built the gradient descent engine for this on Day 3, but today we look at it strictly as a probabilistic classification model, not just a sklearn class. Logistic regression is a strong baseline because it is fast, relatively interpretable, and produces probabilities.

- **The Log-Odds Engine:** Under the hood, log-odds transform probability into an unbounded linear quantity. The algorithm draws a straight line through these log-odds.
- **The Probability Squasher:** That unbounded linear score is transformed into a probability using the sigmoid.
- **The Penalty:** We evaluate the model using binary cross-entropy, which strongly penalizes confident wrong predictions. We also apply regularization to control model complexity.
- **The Threshold:** Because probability is continuous, class prediction is obtained by applying a decision threshold. We will manually manipulate this threshold to alter business outcomes.
- **Expansion:** While we focus on binary outcomes (0 or 1), multiclass classification can be handled with appropriate multiclass strategies.

Phase 2: The Hidden Math (Decision Trees for Classification)

Instead of drawing lines through data, a Decision Tree asks a series of "Yes/No" questions to split the data into branches. To figure out the *best* question to ask, the algorithm uses these exact mathematical formulas to measure "Impurity" (how mixed up the data is).

1. Measuring the Mess (Impurity Metrics):

- **Gini Impurity:** Measures how often a randomly chosen element would be incorrectly labeled. The formula is $Gini = 1 - \sum_{i=1}^C p_i^2$. For the binary case, it simplifies to $Gini = 1 - (p_0^2 + p_1^2)$.
- **Entropy:** An alternative to Gini that measures the level of chaos or unpredictability. The formula is $H = -\sum_{i=1}^K p_i \log_2(p_i)$. For binary classification, it is $H = -p_0 \log_2(p_0) - p_1 \log_2(p_1)$.

2. Splitting the Data:

- **Weighted Gini after splitting:** Once a tree splits the data into Left (L) and Right (R) branches, it calculates the new overall impurity: $Gini_{\text{split}} = \frac{n_L}{n} Gini_L + \frac{n_R}{n} Gini_R$. The tree wants a lower resulting impurity.
- **Information Gain:** If the tree uses Entropy, it measures how much "chaos" was removed by the split: $IG = H(\text{parent}) - \sum \frac{n_i}{n} H(\text{child}_i)$. For two children, this is $IG = H(\text{parent}) - \left[\frac{n_L}{n} H(L) + \frac{n_R}{n} H(R) \right]$. The tree wants to maximize information gain.

Phase 3: Decision Trees for Regression & Complexity

Trees are not just for classification; they can predict continuous numbers too.

- **Regression Tree — MSE:** For a node, the Mean Squared Error is calculated as $MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2$. A regression tree searches for splits that reduce the prediction error/variance inside the resulting nodes.
- **Regression Tree — Variance:** The variance formula is identical to the MSE formula in this context: $Var(Y) = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2$.

- **Tree Complexity:** You should understand the relationship between: Tree depth $\rightarrow$ model complexity $\rightarrow$ bias/variance. Increasing tree depth generally decreases bias, but can cause variance to increase, which can lead to overfitting.

Phase 4: Implementation Pattern & Deliverables

We will run a strict pipeline: $X/y \rightarrow \text{split} \rightarrow \text{preprocessing} \rightarrow \text{baseline} \rightarrow \text{fit} \rightarrow \text{probability predictions} \rightarrow \text{threshold} \rightarrow \text{class predictions} \rightarrow \text{metrics} \rightarrow \text{error analysis}$.

For our deliverables today, we will apply this pipeline directly to a Customer churn prediction project. We will set up the architecture to predict which players are likely to permanently abandon the Retro Reload storefront.

Experiments we will execute:

- Implement sigmoid and log loss with NumPy.
- Inspect coefficients and their direction.
- Compare probability outputs with hard class outputs.
- Change the classification threshold without retraining.

Your End-of-Day Deliverables:

- A Customer churn prediction script.
- A Probability-vs-threshold experiment.
- A Confusion matrix at multiple thresholds.